

---

• • •

LIBRO DI DOCUMENTAZIONE TECNICA

# MasterRent

Portale per affitti universitari  
nella città de L'Aquila

## GRUPPO DI LAVORO

Ramondo Mattia · 291659

Odoardi Davide · 292216

Marinucci Alessandro · 261682

Corso: Tecnologie del Web · A.A. 2025/2026

APERTURA  
00

# Descrizione del progetto

Documentazione finale del progetto del corso di Tecnologie del Web. Portale di affitti universitari per la città de L'Aquila, sviluppato in due fasi: una fase tradizionale/manuale e una fase ri-sviluppata con l'assistenza di strumenti di intelligenza artificiale (LLM).

| Voce                | Valore                                                   |
|---------------------|----------------------------------------------------------|
| Titolo del progetto | MasterRent — Portale per affitti universitari a L'Aquila |
| Corso               | Tecnologie del Web                                       |
| Anno accademico     | 2025 / 2026                                              |
| Tipo di elaborato   | Applicazione web full-stack con relazione comparativa    |
| Stack tecnologico   | PHP 8.x, MySQL/MariaDB, HTML5, CSS3, JavaScript vanilla  |
| Motore di template  | template2.inc.php (nessun framework)                     |

## Gruppo di lavoro

| Componente           | Matricola |
|----------------------|-----------|
| Ramondo Mattia       | 291659    |
| Odoardi Davide       | 292216    |
| Marinucci Alessandro | 261682    |





## SOMMARIO

## 00

# Indice dei capitoli

|    |                                                           |
|----|-----------------------------------------------------------|
| 01 | Abstract                                                  |
| 02 | Introduzione al dominio degli affitti universitari        |
| 03 | Requisiti del corso e copertura nel progetto              |
| 04 | Visione generale dell'applicazione                        |
| 05 | Ruoli: visitatore, studente, proprietario, amministratore |
| 06 | Casi d'uso principali                                     |
| 07 | Architettura generale PHP senza framework                 |
| 08 | Architettura di Fase 1                                    |
| 09 | Architettura di Fase 2                                    |
| 10 | Confronto Fase 1 / Fase 2                                 |
| 11 | Database e tabelle                                        |
| 12 | Diagramma ER                                              |
| 13 | Modello users-groups-services                             |
| 14 | Sessioni, autenticazione, autorizzazioni                  |
| 15 | Sicurezza                                                 |
| 16 | Frontend pubblico                                         |
| 17 | Area studente                                             |
| 18 | Area proprietario                                         |

|           |                                                  |
|-----------|--------------------------------------------------|
| <b>19</b> | Area amministratore                              |
| <b>20</b> | Flusso richiesta visita, caparra e "La mia casa" |
| <b>21</b> | Ricerca e filtri                                 |
| <b>22</b> | Gestione immagini                                |
| <b>23</b> | UI/UX e differenze grafiche tra le due fasi      |
| <b>24</b> | JavaScript vanilla                               |
| <b>25</b> | Installazione ed esecuzione                      |
| <b>26</b> | Credenziali demo                                 |
| <b>27</b> | Testing e verifiche manuali                      |
| <b>28</b> | Diario di sviluppo                               |
| <b>29</b> | Log prompt e uso degli LLM                       |
| <b>30</b> | Analisi dei commit                               |
| <b>31</b> | Limiti noti                                      |
| <b>32</b> | Conclusioni                                      |
| <b>33</b> | Appendici                                        |

# CAPITOLO 01

## Abstract

MasterRent è un portale web pensato per mettere in contatto studenti universitari e proprietari di immobili nella città de L'Aquila.

Il problema di partenza è concreto e locale: dopo il sisma del 2009 e con la progressiva ripresa dell'Università degli Studi dell'Aquila, il mercato degli affitti studenteschi è frammentato tra annunci cartacei, gruppi social e passaparola, con poca trasparenza su prezzi, distanze dai poli didattici e affidabilità delle parti. MasterRent propone un catalogo strutturato di stanze e posti letto, una ricerca per zona, polo universitario, prezzo, tipologia e accessori, e un ciclo completo di interazione che va dalla richiesta di visita fino al versamento simulato della caparra e alla gestione della propria casa.

L'elemento che rende il progetto interessante dal punto di vista didattico non è soltanto l'applicazione, ma il metodo con cui è stata costruita. MasterRent è stato realizzato in due fasi distinte. Nella **Fase 1** il gruppo ha sviluppato il portale in modo tradizionale, scrivendo a mano schema SQL, autenticazione, modello di permessi, CRUD e flussi applicativi. Nella **Fase 2** lo stesso prodotto è stato ripensato e ricostruito con l'ausilio di modelli linguistici di grandi dimensioni (Claude Code, Codex, Antigravity/Gemini e altri strumenti), allo scopo di misurare che cosa cambia in termini di velocità, qualità, organizzazione del codice e affidabilità quando parte del lavoro viene delegata a un assistente AI.

Questo libro documenta entrambe le fasi. Descrive l'architettura di un'applicazione PHP senza framework costruita attorno al motore di template **template2.inc.php**, il modello di autorizzazione **users-groups-services** richiesto dal corso, le diciotto tabelle del database, i

### Tesi del progetto

L'LLM è un acceleratore prezioso su refactoring, interfaccia e documentazione, ma **non sostituisce la comprensione del dominio** né la responsabilità di verifica, che restano interamente umane.

flussi funzionali per i quattro ruoli previsti e le scelte di sicurezza adottate.



## CAPITOLO 02

# Introduzione al dominio degli affitti universitari

## 2.1 Il contesto de L'Aquila

L'Aquila è una città universitaria di medie dimensioni in cui la domanda abitativa studentesca ha caratteristiche particolari. I poli didattici non sono concentrati in un unico campus ma distribuiti: il polo umanistico e i servizi per il diritto allo studio gravitano sul centro storico, il polo scientifico si trova a Coppito (nell'area dell'ospedale), Ingegneria e le facoltà tecniche insistono su Roio.

Questa dispersione geografica rende la variabile "distanza dal proprio polo" almeno tanto importante quanto il prezzo: una stanza economica ma lontana dalla sede frequentata può risultare, nei fatti, più costosa in tempo e trasporti di una stanza più cara ma vicina.

Il dominio applicativo di MasterRent nasce da questa osservazione. Il portale non tratta gli immobili come punti astratti su una mappa, ma li mette in relazione esplicita con i **poli universitari**, memorizzando per ogni coppia immobile-polo una distanza in minuti e un mezzo di trasporto (a piedi, in autobus, in auto).

## 2.2 Gli attori del mercato

Nel mercato reale degli affitti studenteschi si incontrano tre categorie di persone. Gli **studenti** cercano una sistemazione per il periodo dei corsi, spesso con contratti transitori, e hanno bisogno di confrontare rapidamente opzioni per prezzo, zona e servizi inclusi. I **proprietari** dispongono di uno o più immobili, talvolta suddivisi in singole stanze affittate a studenti diversi, e vogliono raggiungere velocemente inquilini affidabili. Esiste infine la necessità di una **figura di governo della piattaforma** che validi i dati, moderi i contenuti e mantenga coerente il catalogo.

MasterRent modella questi tre attori come gruppi applicativi (**student**, **landlord**, **admin**) e aggiunge il visitatore non autenticato come quarto attore implicito, quello che può consultare il catalogo pubblico ma non interagire.

## 2.3 Il ciclo di vita di un affitto nel portale

La logica di dominio ricostruisce un ciclo di vita realistico e semplificato. Lo studente individua una stanza e richiede una visita; il proprietario approva o rifiuta; se approva, lo studente versa

una caparra simulata pari a una mensilità; la stanza passa allo stato **reserved** e diventa la "casa attuale" dello studente; alla conclusione del rapporto la stanza viene liberata e lo studente, avendo effettivamente soggiornato, può lasciare una recensione.

## 2.4 Perché un pagamento simulato

Un portale che gestisce caparre toccherebbe, nel mondo reale, temi di pagamento, antiriciclaggio e tutela del consumatore. Per un progetto didattico questi aspetti sono fuori scope e potenzialmente rischiosi. MasterRent adotta quindi una **caparra simulata**: il flusso di pagamento esiste a livello di stato e di interfaccia, ma non viene mai raccolto né trasmesso alcun dato reale di pagamento.

**CAPITOLO**  
**03**

## Requisiti del corso e copertura nel progetto

Il corso di Tecnologie del Web richiede un insieme preciso di requisiti funzionali e strutturali. La tabella seguente mette a confronto ciascun requisito con la sua realizzazione concreta nella repository.

| #  | Requisito del corso            | Copertura in MasterRent            | Riferimenti reali                      |
|----|--------------------------------|------------------------------------|----------------------------------------|
| 1  | Almeno 14 tabelle SQL          | Superato: 18 tabelle               | sql/001–sql/010, cap. 11               |
| 2  | Modello users-groups-services  | 5 tabelle + require_service()      | 001_auth_schema.sql, permissions.php   |
| 3  | Gestione delle sessioni        | Sessione sicura server-side        | start_secure_session() in security.php |
| 4  | Separazione frontend / backend | Aree distinte, campo services.area | public/, services.area                 |
| 5  | Dashboard admin con CRUD       | CRUD su 8 entità di sistema        | public/admin/**                        |
| 6  | Diagramma ER                   | Fornito in 4 formati               | docs/ER_DIAGRAM.md/.mmd/.svg/.png      |
| 7  | Relazione comparativa          | Nove sezioni obbligatorie          | docs/RELAZIONE_COMPARATIVA.md          |
| 8  | Diario di sviluppo             | Consolidato dai commit             | docs/DIARIO_SVILUPPO.md                |
| 9  | Log dei prompt                 | Documentato con screenshot         | docs/LOG_PROMPT.md                     |
| 10 | Screenshot conversazioni LLM   | 24 immagini indicizzate            | docs/prompt_screenshots/               |
| 11 | Screenshot applicazione        | Guida di cattura predisposta       | docs/APP_SCREENSHOT_GUIDE.md           |
| 12 | Stack senza framework          | PHP procedurale + template2        | template2.inc.php, includes/           |





# CAPITOLO 04

## Visione generale dell'applicazione

MasterRent è un'applicazione web server-side classica: ogni richiesta HTTP viene gestita da uno script PHP nella cartella **public/**, che carica il contesto applicativo tramite **includes/bootstrap.php**, esegue la logica necessaria e produce HTML attraverso il motore di template **template2.inc.php**. Non esiste un front-controller unico né un router: la struttura delle URL corrisponde alla struttura delle cartelle.

Concettualmente l'applicazione si divide in tre livelli. Il **livello di presentazione** è costituito dai file HTML in **templates/** e dal foglio di stile **public/assets/css/style.css**. Il **livello dei controller** è costituito dagli script in **public/**. Il **livello di dominio e accesso ai dati** vive nella cartella **includes/**.

### Mapa dei percorsi principali

| Area              | Percorso            | Ruolo    | Descrizione                                 |
|-------------------|---------------------|----------|---------------------------------------------|
| Home              | public/index.php    | pubblico | Vetrina, ricerca rapida, stanze in evidenza |
| Ricerca           | public/search.php   | pubblico | Catalogo filtrabile delle stanze            |
| Dettaglio         | public/room.php     | pubblico | Scheda stanza, galleria, recensioni         |
| Login             | public/login.php    | pubblico | Autenticazione                              |
| Registrazione     | public/register.php | pubblico | Creazione account                           |
| Area studente     | public/account/**   | student  | Preferiti, prenotazioni, "La mia casa"      |
| Area proprietario | public/landlord/**  | landlord | Immobili, stanze, richieste, upload         |
| Area admin        | public/admin/**     | admin    | CRUD di sistema e moderazione               |
| Prenotazione      | public/booking.php  | student  | Richiesta di visita                         |
| Caparra           | public/deposit.php  | student  | Versamento caparra simulata                 |

Questa organizzazione, volutamente priva di framework, rende evidente il percorso di ogni richiesta e permette di studiare in modo diretto il rapporto tra URL, controller, permesso, query e template — che è precisamente uno degli obiettivi didattici del corso.

# CAPITOLO 05

## Ruoli: visitatore, studente, proprietario, amministratore

MasterRent riconosce quattro ruoli. Tre corrispondono a gruppi applicativi memorizzati nella tabella **user\_groups**; il quarto, il visitatore, è lo stato di default di chi non ha effettuato l'accesso.

### Visitatore

Utente anonimo. Consulta home, ricerche e dettaglio stanza, legge le recensioni pubblicate, ma non salva preferiti né accede ad aree riservate. Ogni tentativo di raggiungere una funzione protetta è intercettato da **require\_login()**.

### Studente (gruppo student)

Utente centrale del portale. Cerca stanze, le salva tra i preferiti, richiede visite, dialoga con il proprietario tramite thread messaggi, versa la caparra simulata, gestisce la propria "casa attuale" e scrive recensioni a rapporto concluso.

### Proprietario (gruppo landlord)

Pubblica immobili e stanze, ne carica le immagini, imposta le distanze dai poli universitari e gestisce le richieste ricevute, approvandole o rifiutandole. Le sue pagine vivono in **public/landlord/**.

### Amministratore (gruppo admin)

Governa la piattaforma. Ha accesso a tutti i CRUD di sistema e agli strumenti di moderazione. Il gruppo **admin** è marcato come **is\_system** nel seed e la sua descrizione recita "Accesso completo".

## Matrice dei permessi essenziali

| Capacità                        | Visitatore | Studente | Proprietario | Admin |
|---------------------------------|------------|----------|--------------|-------|
| Consultare catalogo e dettaglio |            |          |              |       |
| Salvare preferiti               | —          |          | —            | —     |

| Capacità                            | Visitatore | Studente | Proprietario | Admin |
|-------------------------------------|------------|----------|--------------|-------|
| Richiedere visita / versare caparra | —          |          | —            | —     |
| Pubblicare immobili e stanze        | —          | —        |              |       |
| Approvare / rifiutare richieste     | —          | —        |              |       |
| CRUD di sistema e moderazione       | —          | —        | —            |       |



**CAPITOLO**  
**06**

## Casi d'uso principali

I casi d'uso di MasterRent si raggruppano per attore. Di seguito i più significativi; il caso d'uso





più articolato — prenotazione con caparra — è trattato in dettaglio nel capitolo 20.

#### UC-1 Ricerca di una stanza

L'utente apre **public/search.php**, imposta i filtri (quartiere, polo, prezzo massimo, tipologia, access e ottiene l'elenco delle stanze corrispondenti. La query è costruita da **rooms\_search()** **includes/catalog.php**.

#### UC-2 Registrazione e accesso

Un nuovo utente si registra da **public/register.php** scegliendo il tipo di account; la password è salvata come hash bcrypt. L'accesso avviene da **public/login.php**, che apre una sessione sicura e reindirizza alla home di ruolo.

#### UC-3 Salvataggio tra i preferiti

Dalla scheda stanza o dai risultati di ricerca lo studente aggiunge o rimuove una stanza dai preferiti. **toggle\_favorite()** aggiorna la tabella **favorites**.

#### UC-4 Richiesta di visita

Lo studente invia una richiesta tramite **public/booking.php**; **booking\_create()** inserisce un record **bookings** con stato iniziale **visit\_requested**.

#### UC-5 Gestione delle richieste

Il proprietario vede le richieste in **public/landlord/bookings.php** e le approva o rifiuta con **booking\_approve()** / **booking\_reject()**.

#### UC-6 Versamento caparra

Dopo l'approvazione lo studente versa la caparra simulata da **public/deposit.php**. **booking\_mark\_deposit\_paid()** porta la prenotazione a **deposit\_paid** e la stanza a **reserved**.

#### UC-7 Gestione "La mia casa"

Lo studente consulta la propria casa attuale in **public/account/my-house.php** e può avviarne il rilascio.







# CAPITOLO 07

## Architettura generale PHP senza framework

### 7.1 Il bootstrap

Ogni pagina dell'applicazione include, come prima istruzione utile, **includes/bootstrap.php**. Questo file è il punto di "accensione" dell'applicazione: definisce le costanti di percorso (**PROJECT\_ROOT**, **CONFIG\_PATH**, **INCLUDES\_PATH**, **TEMPLATES\_PATH**), carica la configurazione, apre la sessione sicura con **start\_secure\_session()**, stabilisce la connessione al database, carica il motore **template2.inc.php** e infine include i moduli di dominio.

### 7.2 Il motore di template

MasterRent non usa un template engine di terze parti ma **template2.inc.php**, il motore fornito nel contesto del corso. La funzione **render\_template()** in **bootstrap.php** istanzia un oggetto Template a partire dal percorso del file **.html**, inietta le variabili e restituisce l'HTML.

### 7.3 La cartella includes/

Il cuore della logica riutilizzabile vive in **includes/**. La suddivisione è per responsabilità:

**security.php** — sessione, CSRF, escaping, redirect, gestione flash

**auth.php** — login, logout, registrazione

**permissions.php** — modello di autorizzazione

**helpers.php** — utilità di presentazione e dominio

**catalog.php** — ricerca stanze

**admin\_data.php** — dati per la dashboard

**engagement.php** — prenotazioni, messaggi, recensioni

**ZoneEstimates.php** — stime di distanza per macrozona

### 7.4 Flusso di una richiesta

Il percorso tipico è lineare e verificabile: il browser chiama uno script in **public/**, lo script include **bootstrap.php**, applica un eventuale controllo con **require\_login()** o **require\_service()**,



invoca una o più funzioni di dominio da **includes/**, e passa i risultati a **render\_template()**. L'HTML prodotto viene inviato al browser, dove il JavaScript vanilla in **public/assets/js/** attiva le interazioni progressive.

# CAPITOLO 08

## Architettura di Fase 1

La Fase 1 corrisponde alla branch `phase1` e al worktree locale `MasterRent`; utilizza il database `masterrent`. È la versione sviluppata in modo tradizionale, senza assistenza di LLM, e rappresenta la base di riferimento su cui è stata poi misurata la Fase 2.

L'impianto è quello descritto nel capitolo 7: PHP procedurale, logica raccolta in funzioni dentro **`includes/`**, controller espliciti in **`public/`**, template HTML in **`templates/`** e un unico foglio di stile **`public/assets/css/style.css`**. Non esiste un layer a oggetti per l'accesso ai dati: le query PDO sono chiamate direttamente dalle funzioni di dominio.

Questa scelta rende il codice immediatamente leggibile — si vede dove nasce ogni query — al

### Valore formativo

Costruire manualmente ogni pezzo — schema SQL delle 18 tabelle, seed dei gruppi e dei servizi, logica di sessione e CSRF, modello `users-groups-services`, CRUD e flussi di engagement — ha reso esplicite le relazioni tra SQL, controller, template e permessi. Questa comprensione è ciò che ha reso poi verificabile e sicura la Fase 2.

prezzo di una maggiore ripetizione, soprattutto nei moduli CRUD dell'area admin.

# CAPITOLO 09

## Architettura di Fase 2

La Fase 2 corrisponde alla branch phase2 e al worktree locale MasterRent; utilizza il database uniaffitti. È la versione ri-sviluppata con l'assistenza di modelli linguistici, mantenendo la parità funzionale con la Fase 1 ma riorganizzando struttura e presentazione.

La differenza architetturale più rilevante è l'introduzione di un **layer a repository** in **src/Repository**: classi come **BookingRepository**, **RoomRepository**, **UserRepository**, **GroupRepository** e **ServiceRepository** incapsulano l'accesso ai dati che in Fase 1 era distribuito nelle funzioni di **includes/**.

La logica di presentazione è resa modulare da **src/components.php** e dai partial in **templates/frontend/\_components/**, mentre il CSS abbandona il file unico a favore di un **design system a token** (**public/assets/css/tokens.css** e file di componente) che introduce anche la **dark mode**. Il JavaScript vanilla è organizzato in **public/assets/js/** per preferiti, filtri,

### Obiettivo dichiarato

Non aggiungere funzionalità ma **ricostruire meglio**: ridurre la duplicazione, migliorare l'esperienza d'uso e documentare in modo più esplicito le scelte di sicurezza.

lightbox, toast e validazione.

# CAPITOLO 10

## Confronto Fase 1 / Fase 2

Il confronto sintetico tra le due architetture è riassunto nella tabella seguente.

| Dimensione         | Fase 1 (phase1)        | Fase 2 (phase2)                  |
|--------------------|------------------------|----------------------------------|
| Metodo di sviluppo | Tradizionale / manuale | Assistito da LLM                 |
| Database           | masterrent             | uniaffitti                       |
| Worktree locale    | MasterRent             | MasterRent                       |
| Accesso ai dati    | Funzioni in includes/  | Classi in src/Repository         |
| Presentazione      | CSS unico style.css    | Design system a token, dark mode |
| Componenti UI      | Template HTML diretti  | Partial in _components/          |
| JavaScript         | Vanilla essenziale     | Vanilla organizzato per feature  |
| Numero tabelle     | 18                     | 18                               |
| Modello permessi   | users-groups-services  | users-groups-services            |
| Parità funzionale  | Riferimento            | Mantenuta                        |

### Lettura corretta del confronto

Non "una fase migliore dell'altra" ma **due modi diversi di arrivare allo stesso dominio**. La Fase 1 ha prodotto comprensione e controllo; la Fase 2 ha prodotto organizzazione e rifinitura più rapidamente, ma solo perché poggiava sulla base costruita a mano.

# CAPITOLO 11

## Database e tabelle

Lo schema di MasterRent è definito dagli script SQL numerati in **sql/**, pensati per essere importati in ordine. Il database contiene **18 tabelle**, ben oltre il minimo di 14 richiesto dal corso. Gli script si raggruppano per dominio.

| Script                       | Contenuto                                          |
|------------------------------|----------------------------------------------------|
| 000_database.sql             | Creazione del database masterrent                  |
| 001_auth_schema.sql          | users, user_groups, services e giunzioni           |
| 002_auth_seed.sql            | Gruppi, servizi e utenti demo                      |
| 003_laquila_geo_schema.sql   | neighborhoods, university_poles                    |
| 004_laquila_geo_seed.sql     | Quartieri e poli reali de L'Aquila                 |
| 005_laquila_geo_services.sql | Servizi e menu legati alla geografia               |
| 006_properties_schema.sql    | properties, rooms, amenities e giunzioni           |
| 007_properties_seed.sql      | Immobili, stanze e accessori demo                  |
| 008_engagement_schema.sql    | bookings, storico, messaggi, preferiti, recensioni |
| 009_engagement_seed.sql      | Dati demo di engagement                            |
| 010_my_house_flow.sql        | Estensione stati e servizio "La mia casa"          |

### Le cinque famiglie di tabelle

**Autorizzazione** — users, user\_groups, services, users\_has\_groups, services\_has\_groups

**Geografia** — neighborhoods, university\_poles

**Catalogo** — properties, rooms, amenities, room\_has\_amenities, property\_has\_poles, property\_images

**Engagement** — bookings, booking\_status\_history, messages, favorites, reviews

## Scelte di schema notevoli

La tabella **users** prevede il **soft-delete** tramite la colonna **deleted\_at**, così che la cancellazione di un account non distrugga lo storico collegato. La tabella **rooms** porta due campi di stato distinti: **is\_available** (flag di pubblicazione) e **status** (*available/reserved/unavailable*), dove **reserved** indica che la caparra è stata versata. La tabella **bookings** codifica l'intero ciclo di prenotazione in un campo **status** a sette valori.

Il vincolo **UNIQUE (room\_id, student\_id)** impedisce richieste duplicate. Il **CHECK (rating BETWEEN 1 AND 5)** su **reviews** garantisce la validità del voto a livello di database. Tutte le tabelle usano **InnoDB** con charset **utf8mb4** e chiavi esterne con azioni referenziali esplicite.



# CAPITOLO 12

## Diagramma ER

Il diagramma entità-relazione è derivato direttamente dagli script SQL reali e non è ricostruito a memoria. È fornito in quattro formati, così da essere utilizzabile sia nella documentazione sia come sorgente rigenerabile.

| Formato        | Scopo                                    |
|----------------|------------------------------------------|
| ER_DIAGRAM.md  | Descrizione tabellare e sorgente Mermaid |
| ER_DIAGRAM.mmd | Sorgente Mermaid isolato                 |
| ER_DIAGRAM.png | Versione raster                          |

### Relazioni principali

Un utente può appartenere a più gruppi e un gruppo raccoglie più utenti (**users\_has\_groups**); un gruppo abilita più servizi e un servizio è concesso a più gruppi (**services\_has\_groups**). Un proprietario possiede più immobili (**properties.landlord\_id**); un immobile appartiene a un quartiere e contiene più stanze. Una stanza espone più accessori tramite **room\_has\_amenities**, e un immobile dichiara le distanze verso più poli tramite **property\_has\_poles**.

Sul versante engagement, una prenotazione lega una stanza a uno studente (**bookings**), genera uno storico (**booking\_status\_history**) e un thread di messaggi (**messages**); i preferiti (**favorites**) e le recensioni (**reviews**) legano anch'essi studente e stanza.

# CAPITOLO 13

## Modello users-groups-services

Il modello di autorizzazione è il requisito architetturale più caratterizzante del corso e MasterRent lo implementa nella sua forma canonica a cinque tabelle, evitando esplicitamente la scorciatoia di un singolo campo *role*.

### Regola esplicita

Il commento in testa a `sql/001_auth_schema.sql` lo dichiara: *"Do not replace this model with a single role column in users"*.

Il funzionamento è a due livelli di indirezione. Gli **utenti** sono associati a **gruppi** tramite **users\_has\_groups**; i **servizi** — che rappresentano rotte, funzioni o voci di menu autorizzabili— sono associati ai gruppi tramite **services\_has\_groups**. Un utente "possiede" un servizio se esiste almeno un gruppo che lo collega a quel servizio.

### Gruppi di sistema

| Codice   | Nome           | Descrizione                                                       |
|----------|----------------|-------------------------------------------------------------------|
| admin    | Amministratori | Accesso completo: utenti, gruppi, servizi, moderazione            |
| landlord | Proprietari    | Proprietari che pubblicano e gestiscono annunci di stanze         |
| student  | Studenti       | Studenti che cercano stanze, salvano preferiti, inviano richieste |

frontend/backend richiesta dal corso, oltre a metadati per la costruzione dei menu (**is\_menu\_item**, **menu\_order**) e per l'attivazione (**is\_active**).

Lato applicativo, le funzioni chiave in **includes/permissions.php** sono **user\_groups()**, **user\_services()**, **has\_service()** e la guardia **require\_service()**. Un controller che debba proteggere una rotta backend invoca **require\_service('...')**; se l'utente corrente non possiede quel servizio, la funzione blocca l'accesso.

Ogni servizio porta un campo **area** (*auth/frontend/backend*) che realizza la separazione

# CAPITOLO 14

## Sessioni, autenticazione, autorizzazioni

### 14.1 Sessione sicura

La sessione viene aperta da **start\_secure\_session()** in **includes/security.php**, richiamata dal bootstrap prima di qualsiasi output. La funzione configura i parametri del cookie di sessione tenendo conto del contesto (per esempio il flag **secure** quando la richiesta è in HTTPS), così da mitigare furto di sessione e trasmissione in chiaro.

### 14.2 Autenticazione

Registrazione e login sono gestiti da **includes/auth.php** e dai controller **public/register.php** e **public/login.php**. Le password non sono mai memorizzate in chiaro: vengono cifrate con l'algoritmo **bcrypt** tramite le funzioni native di PHP e salvate in **users.password\_hash**. Il logout avviene via POST attraverso **public/logout.php**, scelta che evita la disconnessione accidentale tramite semplici link o prefetch del browser.

### 14.3 Autorizzazione

L'autorizzazione poggia interamente sul modello **users-groups-services**. I controller riservati invocano **require\_login()** per l'autenticazione e **require\_service()** — o, dove opportuno, **user\_has\_group()** — per l'autorizzazione puntuale. La combinazione garantisce che un proprietario non possa accedere alle pagine dell'area amministrativa anche se autenticato.

**CAPITOLO**  
**15****Sicurezza**

La sicurezza di MasterRent non è affidata a un singolo meccanismo ma a una serie di difese





sovrapposte, presenti in entrambe le fasi e centralizzate in **includes/security.php**.

#### Iniezioni SQL

Tutte le query passano da statement **PDO preparati** con parametri legati: nessun valore di input viene concatenato nel testo SQL.

#### Cross-site scripting (XSS)

L'output verso il template è sistematicamente filtrato con **htmlspecialchars**, incapsulato nella funzione **htmlspecialchars()**.

#### Cross-site request forgery (CSRF)

Le POST che modificano lo stato sono protette da token: **csrf\_token()**, **csrf\_field()**, **verify\_csrf\_token()**, con scope distinti per form diversi.

#### Messaggi flash

**set\_flash()**, **take\_flashes()**, **render\_flashes()** evitano di trasportare stato in URL.

#### Accessi e ownership

**require\_login()** e **require\_service()** proteggono le rotte; controlli di ownership impediscono che un proprietario agisca su immobili non suoi (mitigazione IDOR).

#### Integrità dei dati

Il soft-delete degli utenti preserva l'integrità referenziale; il versamento della caparra aggiorna **bookings**, **booking\_status\_history** e **rooms.status** in **transazione**.

#### Upload immagini

**save\_property\_image\_upload()** valida i file prima del salvataggio in **public/assets/uploads/**.

#### Audit di sicurezza (Fase 2)

SECURITY\_AUDIT.md nel worktree Fase 2 ha rafforzato upload, logout POST e controlli di owners



# CAPITOLO 16

## Frontend pubblico

Il frontend pubblico è la parte del portale accessibile senza autenticazione: home, ricerca, dettaglio stanza e le pagine di ingresso.

### Home

**public/index.php**, template **templates/home.html**. Svolge il ruolo di vetrina: presenta il portale, offre una barra di ricerca rapida e mostra una selezione di stanze in evidenza e delle recensioni più recenti pubblicate (**reviews\_latest\_published()**).

### Ricerca

**public/search.php**, template **templates/search.html**. È il catalogo filtrabile, trattato nel capitolo 21.

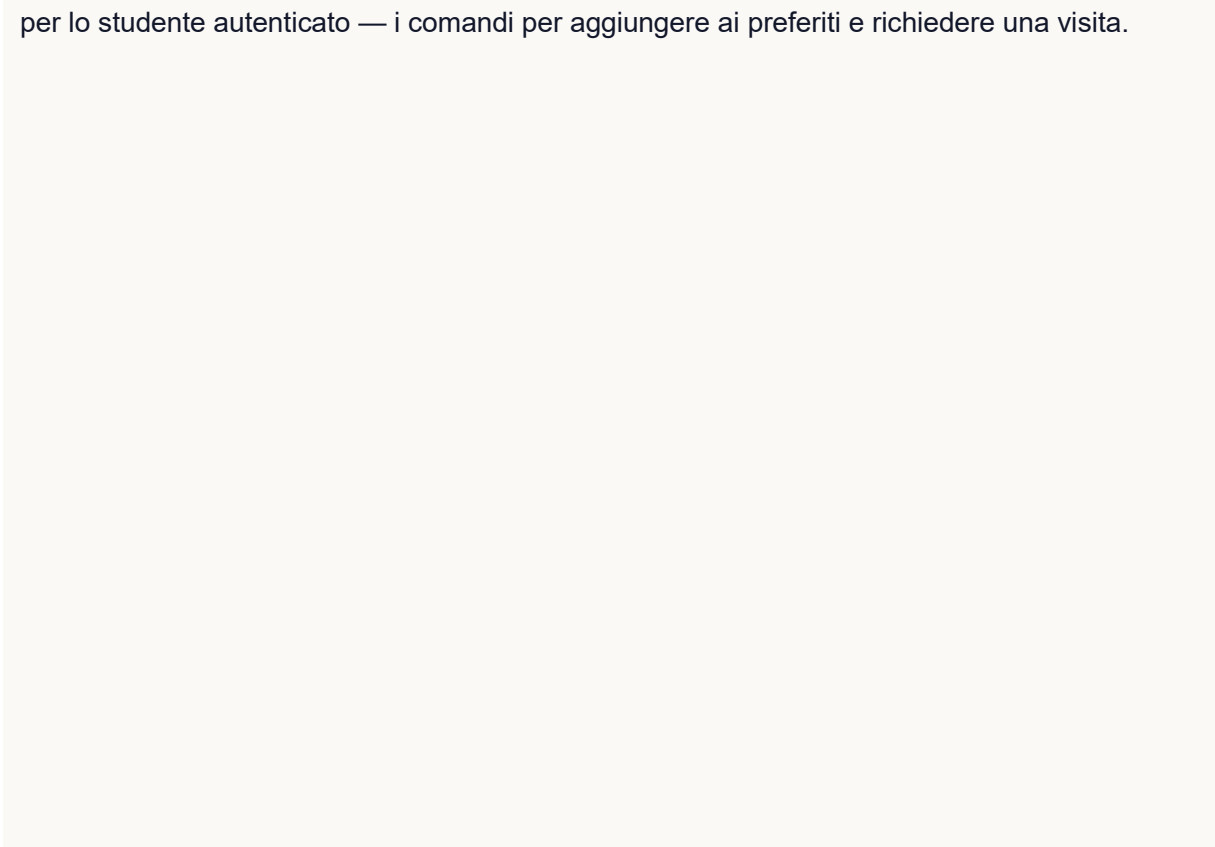
### Dettaglio stanza

**public/room.php**, template **templates/room.html**. Mostra la galleria delle immagini, le informazioni sull'immobile e sulla stanza, le distanze dai poli, gli accessori, le recensioni e —

#### Screenshot da inserire

Le schermate di home, ricerca, dettaglio stanza e login non sono ancora presenti nella repository. La guida **APP\_SCREENSHOT\_GUIDE.md** indica per ciascuna il nome file atteso in **docs/app\_screenshots/**.

per lo studente autenticato — i comandi per aggiungere ai preferiti e richiedere una visita.



# CAPITOLO 17

## Area studente

L'area studente vive in **public/account/** ed è accessibile agli utenti del gruppo **student**. Il cruscotto (**public/account/index.php**) offre l'accesso alle sottosezioni: profilo, preferiti, prenotazioni e "La mia casa".

### Profilo

**public/account/profile.php** permette di aggiornare i dati anagrafici e la password.

### Preferiti

**public/account/favorites.php** elenca le stanze salvate; la gestione passa da **toggle\_favorite()** e dalla tabella **favorites**. La lista è resa con **room\_cards\_html()**, la stessa funzione usata in ricerca, per garantire coerenza visiva.

### Prenotazioni

**public/account/bookings.php** mostra lo stato di ciascuna richiesta con badge e stepper (**booking\_status\_badge()**, **booking\_stepper\_html()**), e dà accesso al thread di messaggi con il proprietario.

### La mia casa

**public/account/my-house.php** è descritta in dettaglio nel capitolo 20.

# CAPITOLO 18

## Area proprietario

L'area proprietario vive in **public/landlord/** ed è riservata al gruppo **landlord**. Il cruscotto (**public/landlord/index.php**) riassume gli immobili pubblicati e le richieste in arrivo.

La gestione dell'immobile passa da **public/landlord/property.php** e dal form **public/landlord/property\_form.php**, mentre le stanze si creano e modificano con **public/landlord/room\_form.php**. Il caricamento delle immagini avviene tramite **public/landlord/upload.php**, che si appoggia a **save\_property\_image\_upload()** per validare e salvare i file in **public/assets/uploads/**.

Le richieste ricevute sono gestite in **public/landlord/bookings.php**: da qui il proprietario approva (**booking\_approve()**) o rifiuta (**booking\_reject()**) una richiesta e dialoga con lo studente. Alla creazione di un immobile il proprietario associa il quartiere e dichiara le distanze verso i poli universitari, popolando **property\_has\_poles**.

CAPITOLO  
19

## Area amministratore

L'area amministrativa vive in **public/admin/** ed è riservata al gruppo **admin**. Rappresenta la dashboard CRUD richiesta dal corso e copre otto entità di sistema, oltre alla moderazione di prenotazioni e recensioni.

| Entità            | Percorso                    | Operazioni                                     |
|-------------------|-----------------------------|------------------------------------------------|
| Utenti            | public/admin/users/         | Elenco, creazione, cancellazione (soft-delete) |
| Gruppi            | public/admin/groups/        | Elenco, creazione, cancellazione               |
| Servizi           | public/admin/services/      | Elenco, creazione, cancellazione               |
| Quartieri         | public/admin/neighborhoods/ | Elenco, creazione, cancellazione               |
| Poli universitari | public/admin/poles/         | Elenco, creazione, cancellazione               |
| Accessori         | public/admin/amenities/     | Elenco, creazione, cancellazione               |
| Immobili          | public/admin/properties/    | Elenco, form, vista, cancellazione             |
| Stanze            | public/admin/rooms/         | Form                                           |
| Prenotazioni      | public/admin/bookings/      | Elenco e moderazione                           |
| Recensioni        | public/admin/reviews/       | Elenco, pubblicazione, occultamento            |

I dati per le viste admin sono forniti da **includes/admin\_data.php**, mentre la moderazione delle recensioni usa **review\_set\_status()** e **review\_delete()** da **includes/engagement.php**. La gestione di utenti, gruppi e servizi è il punto in cui il modello **users-groups-services** diventa operativo dall'interfaccia.

# CAPITOLO 20

## Flusso richiesta visita, caparra e "La mia casa"

Questo è il caso d'uso centrale di MasterRent e attraversa più tabelle e più ruoli.

Lo stato di una prenotazione è codificato nel campo **bookings.status**, che assume sette valori: **visit\_requested**, **approved\_pending\_deposit**, **rejected**, **deposit\_paid**, **cancellation\_requested**, **completed**, **withdrawn**. Ogni transizione è registrata in **booking\_status\_history** da **booking\_add\_history()**, con nota e autore.

### Le fasi del flusso

1) **Studente** richiede una visita da **public/booking.php**: **booking\_create()** crea il record con stato **visit\_requested**.

2) **Proprietario** valuta la richiesta in **public/landlord/bookings.php**: se la rifiuta, **booking\_reject()** porta lo stato a **rejected**; se la approva, **booking\_approve()** lo porta a **approved\_pending\_deposit**.

3) **Studente** versa la caparra simulata da **public/deposit.php**: **booking\_mark\_deposit\_paid()** registra importo, data e riferimento fittizio, porta la prenotazione a **deposit\_paid** e, nella stessa transazione, aggiorna **rooms.status** a **reserved**.

#### Regola caparra & "casa attuale"

La caparra è pari a una mensilità, calcolata da **deposit\_amount\_for()**. Nessun dato reale di pagamento viene raccolto. Una regola di dominio impedisce a uno studente di prenotare o versare una nuova caparra se ha già una casa attuale: **booking\_active\_for\_student()** verifica questa condizione.

### Transizioni principali

| Stato di partenza | Azione           | Attore       | Stato di arrivo          | Funzione          |
|-------------------|------------------|--------------|--------------------------|-------------------|
| —                 | Richiesta visita | Studente     | visit_requested          | booking_create()  |
| visit_requested   | Approvazione     | Proprietario | approved_pending_deposit | booking_approve() |

| Stato di partenza        | Azione             | Attore         | Stato di arrivo         | Funzione                       |
|--------------------------|--------------------|----------------|-------------------------|--------------------------------|
| visit_requested          | Rifiuto            | Proprietario   | rejected                | booking_reject()               |
| approved_pending_deposit | Caparra versata    | Studente       | deposit_paid + reserved | booking_mark_deposit_paid()    |
| deposit_paid             | Richiesta disdetta | Studente       | cancellation_requested  | booking_request_cancellation() |
| deposit_paid/canc.req    | Rilascio stanza    | Studente/Prop. | completed/withdrawn     | booking_release_room()         |

Dopo il pagamento, la stanza diventa la "casa attuale" dello studente, consultabile in **public/account/my-house.php**. Quando il rapporto si conclude, **booking\_release\_room()** riporta **rooms.status** ad **available**. Solo dopo un rapporto **completed** lo studente acquisisce il diritto di recensire, verificato da **review\_student\_stayed()**.

## CAPITOLO 21

# Ricerca e filtri

La ricerca è la funzione più usata del portale ed è realizzata dal controller **public/search.php** con la funzione di dominio **rooms\_search()** in **includes/catalog.php**. La funzione accetta un array di filtri e costruisce dinamicamente la query: aggiunge una condizione solo per i filtri effettivamente valorizzati, evitando di penalizzare le ricerche ampie.

## Filtri disponibili

**G** **Quartiere** — dalla tabella **neighborhoods**

**G** **Polo universitario** di riferimento — tramite **property\_has\_poles**

**G** **Prezzo massimo** mensile

**G** **Tipologia** di stanza — single, double, bed\_space, entire\_apartment

**G** **Accessori** — via **room\_has\_amenities**

I risultati sono resi con **room\_cards\_html()**, che produce le card e, per lo studente autenticato, evidenzia le stanze già presenti tra i preferiti passando l'elenco restituito da

### Distintivo di MasterRent

La relazione con i poli è ciò che distingue MasterRent da un generico catalogo immobiliare: la scelta del polo non filtra soltanto per prossimità geografica ma sfrutta le **distanze reali** memorizzate per coppia immobile-polo. Il modulo **includes/ZoneEstimates.php** fornisce inoltre stime indicative per macrozona (Centro, Torrione/Strinella, Coppito/Ospedale).



current\_favorite\_ids().

# CAPITOLO 22

## Gestione immagini

Le immagini degli annunci sono gestite dalla tabella **property\_images**, che lega ogni file a un immobile e distingue l'immagine di copertina tramite il flag **is\_cover**, con un'eventuale didascalia. Il caricamento avviene dall'area proprietario (**public/landlord/upload.php**) e passa per **save\_property\_image\_upload()** in **includes/helpers.php**, che valida il file e lo salva in **public/assets/uploads/**. La rimozione fisica del file è gestita da **delete\_uploaded\_image\_file()**.

La resa lato interfaccia è centralizzata in due helper: **property\_image\_public\_url()** costruisce l'URL pubblico a partire dal nome file, e **property\_image\_markup()** produce il markup **<img>** completo di attributo **alt** e classe CSS.

Le immagini demo degli immobili sono versionate nella repository sia in formato **.avif** (cartella **Case/** e **public/assets/uploads/case/**) sia come caricamenti **.jpg** di esempio. La scelta di usare foto locali reali, invece di immagini remote, è documentata in **docs/IMAGE\_SOURCES.md** e rende l'applicazione autoconsistente anche offline.

# CAPITOLO 23

## UI/UX e differenze grafiche tra le due fasi

Le due fasi condividono le stesse funzionalità ma offrono esperienze visive diverse, e questa differenza è uno dei risultati più tangibili del confronto.

### Fase 1 — Essenzialità funzionale

Interfaccia essenziale e tradizionale: un unico foglio di stile `public/assets/css/style.css`, form e pagine renderizzate lato server, interazioni JavaScript ridotte al minimo indispensabile. L'obiettivo era la chiarezza e la copertura funzionale, non la rifinitura estetica.

### Fase 2 — Design system

Introduzione di un vero e proprio design system. Il CSS viene scomposto in **token** (`public/assets/css/tokens.css`) e componenti, il che consente coerenza cromatica e tipografica e, soprattutto, l'introduzione della **dark mode**. Card degli annunci, galleria con lightbox, toast di notifica, stepper del flusso di prenotazione e barra di ricerca sticky su mobile segnano il salto di qualità percepita.

| Aspetto UI         | Fase 1               | Fase 2                |
|--------------------|----------------------|-----------------------|
| Foglio di stile    | Unico (style.css)    | Token + componenti    |
| Dark mode          | Assente              | Presente              |
| Card annunci       | Statiche             | Interattive, uniformi |
| Galleria immagini  | Immagini in sequenza | Lightbox              |
| Notifiche          | Flash server-side    | Flash + toast JS      |
| Stato prenotazione | Etichetta            | Stepper visuale       |
| Ricerca su mobile  | Standard             | Sticky / ottimizzata  |

# CAPITOLO 24

## JavaScript vanilla

Coerentemente con il vincolo "nessun framework", MasterRent non usa librerie front-end: tutte le interazioni sono realizzate in JavaScript vanilla, servito da **public/assets/js/**. Il principio adottato è quello del *progressive enhancement*: le funzioni fondamentali restano disponibili anche senza JavaScript, perché il rendering è server-side, mentre lo script aggiunge comodità e reattività.

### Interazioni principali

- g Toggle dei preferiti senza ricaricare la pagina — dialoga con **toggle\_favorite()**
- g Comportamento dei filtri di ricerca
- g Galleria con lightbox nel dettaglio stanza
- g Toast di conferma delle azioni
- g Validazione dei form lato client — **affianca**, non sostituisce, la validazione lato server

La Fase 2 ha organizzato questo codice per funzionalità, rendendolo più leggibile e riusabile rispetto agli script più essenziali della Fase 1.

# CAPITOLO 25

## Installazione ed esecuzione

MasterRent è pensato per un ambiente **XAMPP** su Windows con **PHP 8.x** e **MySQL/MariaDB**. Le due fasi vivono in worktree e database distinti.

### Fase 1 — branch phase1, database masterrent

Importare gli script SQL in ordine numerico e avviare il server PHP integrato:

```
# Import SQL
sql/000_database.sql
sql/001_auth_schema.sql
sql/002_auth_seed.sql
sql/003_laquila_geo_schema.sql
sql/004_laquila_geo_seed.sql
sql/005_laquila_geo_services.sql
sql/006_properties_schema.sql
sql/007_properties_seed.sql
sql/008_engagement_schema.sql
sql/009_engagement_seed.sql
sql/010_my_house_flow.sql
```

```
# Avvio server
cd C:\Users\Ospite\Desktop\MasterRent
C:\xampp\php\php.exe -S 127.0.0.1:8000 -t public
```

Il portale è quindi raggiungibile su **http://127.0.0.1:8000/**.

### Fase 2 — branch phase2, database uniaffitti

Il worktree MasterRent usa una numerazione SQL propria (00–09) e la porta 8002. La procedura di dettaglio è nel README.md della repository.

## Configurazione

La configurazione è in **config/config.php** (nome applicazione, parametri di connessione, BASE\_URL/ASSETS\_URL rilevati automaticamente) e in **config/database.php** (connessione PDO). Nessuna credenziale reale è necessaria oltre all'utente **root** locale.

**CAPITOLO**  
**26**

## Credenziali demo

Il seed popola alcuni account di prova, tutti con la medesima password. Sono pensati per verificare rapidamente i tre ruoli.

**Password comune per tutti gli account demo**`Admin123!`

| Ruolo        | Email                     |
|--------------|---------------------------|
| Admin        | admin@uniaffitti.local    |
| Proprietario | odo@uniaffitti.local      |
| Proprietario | laura@uniaffitti.local    |
| Studente     | studente@uniaffitti.local |
| Studente     | giulia@uniaffitti.local   |
| Studente     | luca@uniaffitti.local     |

Gli indirizzi usano il dominio storico **uniaffitti.local**, coerente con il naming tecnico del progetto; non corrispondono ad account reali.

**CAPITOLO**  
**27**

## Testing e verifiche manuali

Il progetto non adotta una suite di test automatici — scelta comune per un elaborato didattico di queste dimensioni — ma si basa su un protocollo di **verifiche manuali** ripetute a ogni fase e su controlli statici del codice PHP.

Sul piano statico, i file PHP toccati sono stati verificati con **php -l** per escludere errori di sintassi. Sul piano funzionale, il protocollo di verifica copre l'intero ciclo applicativo.

### Controlli chiave

| Verifica                                                                         | Esito atteso                                   |
|----------------------------------------------------------------------------------|------------------------------------------------|
| Import SQL 000–010                                                               | Database masterrent popolato senza errori      |
| Login tre ruoli                                                                  | Redirect alla home di ruolo corretta           |
| Ricerca con filtri                                                               | Risultati coerenti con i filtri                |
| Richiesta <input type="checkbox"/> approvazione <input type="checkbox"/> caparra | Stanza passa a reserved                        |
| Vincolo casa attuale                                                             | Blocco di una seconda prenotazione             |
| Rilascio stanza                                                                  | Stanza torna available, prenotazione completed |
| Recensione senza soggiorno                                                       | Operazione negata                              |
| CRUD admin                                                                       | Creazione/cancellazione riflesse nel DB        |

# CAPITOLO 28

## Diario di sviluppo

Il diario completo e consolidato è mantenuto in **DIARIO\_SVILUPPO.md**; qui se ne riporta la sintesi cronologica, ricostruita dai commit reali e dagli script SQL.

### Fase 1

Parte dall'inizializzazione del progetto (maggio 2026), prosegue con l'organizzazione della struttura, il database, il bootstrap e il login, poi con il pannello admin e l'area proprietario. La baseline vera e propria della branch **phase1** è stata raggiunta all'inizio di luglio 2026. Sono poi seguiti credenziali demo, semplificazione del frontend, flussi e catalogo, moduli admin e primo diagramma ER. Il 7 luglio la Fase 1 riceve l'allineamento con la Fase 2, a cui sono seguiti i deliverable documentali richiesti dalla consegna.

### Fase 2

Condivide la storia iniziale con la Fase 1 e prosegue sulla branch **phase2** con interventi su interfaccia,

#### Lezione trasversale

L'ordine — **prima Fase 1 manuale, poi Fase 2 assistita** — non è casuale: la comprensione costruita a mano nella prima fase è ciò che ha reso verificabile e sicura la seconda.



immagini, mappa, preferiti, requisiti di consegna e correzioni finali.

# CAPITOLO 29

## Log prompt e uso degli LLM

Il log completo dei prompt è in **LOG\_PROMPT.md** e l'indice delle immagini in **SCREENSHOT\_INDEX.md**. La repository conserva **24 screenshot autentici** delle sessioni con gli strumenti LLM, in **docs/prompt\_screenshots/**, organizzati per strumento.

### Strumenti impiegati

| Strumento               | Uso documentato                                                |
|-------------------------|----------------------------------------------------------------|
| Claude Code (CLI e app) | Build a fette, repository, CRUD, restyling, verifiche          |
| Codex                   | Correzioni UI, home, immagini, dark mode, flusso "La mia casa" |
| Antigravity / Gemini    | Correzioni dominio annunci, quartieri, vincoli su casa attuale |
| ChatGPT web / Chrome    | Redazione di prompt tecnici poi incollati in Codex             |
| Lovable                 | Supporto alla direzione estetica della Fase 2                  |



# CAPITOLO 30

## Analisi dei commit

L'analisi si basa sul log Git reale, riportato integralmente in **COMMIT\_LOG.md**; non riscrive la storia né attribuisce commit ai singoli componenti oltre a quanto Git effettivamente mostra.

La storia si articola su tre riferimenti. La **base comune** (maggio 2026) porta l'inizializzazione, l'organizzazione della struttura, il primo database/login e i primi pannelli admin. Da qui divergono la branch **phase1**, che raccoglie lo sviluppo tradizionale e i documenti finali, e la branch **phase2**, che raccoglie lo sviluppo assistito da LLM. Il ref **main**

resta la base condivisa.

### **Nota metodologica**

Gli autori Git tecnici (odoardy, Mattia, master) non permettono di attribuire in modo affidabile i commit ai tre componenti del gruppo. Se il lavoro è stato svolto in coppia/gruppo o con account condivisi, va dichiarato così nella consegna. Il file **PIANO\_COMMIT.md** va letto come ricostruzione organizzativa, non come log reale.

CAPITOLO  
**31**

## Limiti noti



Il progetto ha limiti dichiarati apertamente, coerenti con la sua natura didattica.

#### **Pagamento caparra simulato**

Non esiste integrazione con alcun gestore di pagamento e nessun dato reale viene trattato.

#### **Nessuna suite di test automatici**

La verifica è manuale, con il rischio di regressioni non intercettate.

#### **Notifiche solo in-app**

Approvazioni e messaggi sono affidati all'interfaccia; nessun canale esterno come email.

#### **Distanze dai poli manuali**

Inserite manualmente dal proprietario (Fase 1) o stimate (Fase 2), non calcolate da un servizio di routing esterno.

#### **Attribuzione commit non ricostruibile**

Dal solo log Git non è possibile attribuire in modo affidabile ogni commit ai singoli componenti.

#### **Doppia identità di naming**

Persiste tra le due fasi (**masterrent/uniaffitti**, **MasterRentPhase1/MasterRentPhase2**): il nome ufficiale resta **MasterRent**.



## CAPITOLO 32

# Conclusioni

MasterRent raggiunge il suo doppio obiettivo: è un portale funzionante che copre e supera i requisiti del corso, ed è al tempo stesso un esperimento controllato sul valore dello sviluppo assistito da LLM.

**Sul piano del prodotto**, l'applicazione implementa il modello **users-groups-services** nella forma canonica, gestisce un dominio realistico con 18 tabelle, copre quattro ruoli e un ciclo completo di prenotazione con caparra simulata, il tutto senza framework, con PHP procedurale e il motore **template2.inc.php**.

**Sul piano del metodo**, il confronto tra le due fasi porta a una conclusione sobria e utile. La Fase 1 manuale è stata più lenta ma ha costruito la comprensione del dominio e il controllo sul codice; la Fase 2 assistita è stata più rapida su refactoring, interfaccia e documentazione, ma efficace solo perché fondata sulla base della prima. Gli LLM si sono rivelati acceleratori potenti e insieme fonti di errori sottili — contesti persi, database confusi, scope ridotti — che soltanto un gruppo competente poteva riconoscere e correggere.

### Il messaggio finale

L'intelligenza artificiale è uno strumento che **moltiplica** la competenza di chi la usa, non un **sostituto** di quella competenza.

# CAPITOLO 33

## Appendici

### Appendice A — Mappa dei file principali

| Categoria            | Percorsi                                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------------------|
| Configurazione       | config/config.php, config/database.php                                                                                   |
| Bootstrap e template | includes/bootstrap.php, template2.inc.php                                                                                |
| Sicurezza e permessi | includes/security.php, includes/permissions.php, includes/auth.php                                                       |
| Dominio              | includes/catalog.php, includes/engagement.php, includes/admin_data.php, includes/helpers.php, includes/ZoneEstimates.php |
| Controller pubblici  | public/index.php, public/search.php, public/room.php, public/login.php, public/register.php                              |
| Aree riservate       | public/account/**, public/landlord/**, public/admin/**                                                                   |
| Template             | templates/**                                                                                                             |
| Schema e seed        | sql/000–sql/010                                                                                                          |

### Appendice B — Rigenerazione del diagramma ER

Il sorgente Mermaid è **docs/ER\_DIAGRAM.mmd**. Per rigenerare le versioni SVG e PNG con la Mermaid CLI (richiede Node.js):

```
npm install -g @mermaid-js/mermaid-cli
```

**mermaid.live** ed esportare SVG/PNG.

### Appendice C — Indice dei documenti collegati

| Documento                  | Scopo                                       |
|----------------------------|---------------------------------------------|
| INDEX.md                   | Indice generale dei materiali di consegna   |
| DOCUMENTAZIONE_PROGETTO.md | Sintesi che rimanda a questo libro          |
| RELAZIONE_COMPARATIVA.md   | Relazione comparativa in nove sezioni       |
| DIARIO_SVILUPPO.md         | Diario di sviluppo consolidato              |
| LOG_PROMPT.md              | Log dei prompt LLM                          |
| SCREENSHOT_INDEX.md        | Indice degli screenshot                     |
| APP_SCREENSHOT_GUIDE.md    | Guida agli screenshot dell'app da catturare |
| COMMIT_LOG.md              | Log commit reale                            |
| ER_DIAGRAM.md              | Diagramma ER e descrizione tabelle          |
| GAP_ANALYSIS.md            | Analisi dei gap tra le fasi                 |
| PARITY_CHECKLIST.md        | Checklist di parità funzionale              |

### Casa attuale

Stanza in stato **reserved** associata allo studente che ha versato la caparra, consultabile in "La mia casa".

### Servizio (service)

Unità autorizzabile del modello **users-groups-services**, corrispondente a una rotta, una funzione o una voce di menu.

**Soft-delete**

Cancellazione logica di un utente tramite **deleted\_at**, senza rimozione fisica del record.

## Appendice D — Glossario

**Caparra simulata**

Importo pari a una mensilità registrato nel flusso di prenotazione senza alcun trattamento di dati di pagamento reali.

**Stepper**

Rappresentazione visuale a fasi dello stato di una prenotazione.

**Fine del libro di documentazione tecnica di MasterRent.**